La boucle `for` de ce code parcourt les valeurs de 1 jusqu'à `n` et à chaque fois qu'une de ces valeurs est impaire, l'instruction `print` située au sein du bloc de code de cette boucle `for` sera ignorée et une nouvelle itération sera considérée.

L'exécution de ce code avec la valeur 10 saisie au clavier donne le résultat ci-dessous :

```
Veuillez saisir un nombre > 1 ? 10
2**2 = 4
4**2 = 16
6**2 = 36
8**2 = 64
10**2 = 100
fin du programme!
```

On voit que les valeurs 3, 5, 7 et 9 ont été ignorées par le programme ! En effet, pour ces valeurs, l'instruction `continue` permet de repartir du début de la boucle `for` en ignorant tout ce qui se trouve après cette clause `continue`.

Remarque

Bien que les clauses `break` et `continue` peuvent s'avérer intéressantes et justifiées, il faut les éviter autant que possible, car elles peuvent introduire de la complexité qui nuit à la bonne lisibilité des programmes. En résumé, `break` et `continue` sont utiles, mais à utiliser avec modération.

4.6 Les fonctions

Nous avons vu que les boucles permettent d'automatiser un traitement dont lequel un bloc de code doit être exécuté plusieurs fois de façon successive. En revanche, comment faire lorsqu'un bloc de code doit être exécuté plusieurs fois, mais pas forcément de manière successive ?

Lorsque les instructions d'un bloc de code apparaissent plusieurs fois dans des endroits différents de notre programme, nous pouvons tout simplement factoriser ces instructions dans une seule et même partie de ce programme. Cette partie du programme où les instructions de plusieurs blocs de code ont été factorisées est appelée une procédure ou un sous-programme.

© Editions ENI - All rights reserved

Cette procédure sera appelée autant de fois que nécessaire à partir d'endroits différents d'un programme. Si l'exécution d'une procédure se termine en retournant une valeur, alors cette procédure sera considérée comme étant une fonction.

■Remarque

Contrairement à certains langages de programmation qui distinguent explicitement le concept de procédure de celui de fonction, comme le langage Pascal, avec le langage Python, ces deux concepts se définissent avec la même syntaxe et les procédures sont considérées comme des fonctions.

Nous avons déjà rencontré et utilisé le concept de fonction depuis le début de ce chapitre. Par exemple, nous avons utilisé la fonction `print` à chaque fois que nous avons affiché les résultats de nos programmes précédents, ou encore nous avons utilisé la fonction `input` pour inviter l'utilisateur à saisir des valeurs à l'aide du clavier. Les fonctions telles que `print` et `input` sont prédéfinies nativement dans le langage Python et c'est bien pour cette raison que pour les utiliser il a suffi de les appeler en utilisant leur nom. Cette section montre justement comment définir vos propres fonctions.

4.6.1 Définir et utiliser une fonction sans paramètre

Le code de l'exemple suivant montre comment définir et appeler une fonction sans paramètre.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *fonction-1.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import random as rd

def print_random_list():

    rd_list = []
    for i in range(5):
        rd_list.append(rd.randint(1,10))

    print(f"rd_list={rd_list}")

rd.seed(1)
print_random_list()
```

Cet exemple commence par importer le module `random`. Ce module permet d'utiliser un générateur de nombres aléatoires.

Le mot-clé `def` permet de déclarer et de définir la fonction `print_random_list`. Remarquez que le nom de cette fonction est suffixé avec les deux parenthèses `()` et que rien n'a été mentionné entre ces parenthèses. Cela signifie que la fonction `print_random_list` n'attend pas de paramètres pour s'exécuter. Les deux points à la fin de la déclaration de cette fonction indiquent que le bloc du code suivant correspond à sa définition.

Le corps de la fonction `print_random_list` commence par la définition de la liste vide `rd_list`. Puis, dans une boucle `for` de cinq itérations, les éléments de cette liste seront générés de manière aléatoire en utilisant la fonction `randint` du module `random`. Chaque élément de la liste `rd_list` sera aléatoire, mais compris dans l'intervalle `[1,10]` qui est défini par les deux arguments de cette fonction `randint`. La dernière instruction du bloc de code de la fonction `print_random_list` permet d'afficher la liste `rd_list`.

Après la définition de la fonction `print_random_list` et de retour au programme principal, l'instruction `rd.seed(1)` permet d'initialiser le générateur de nombres aléatoires afin que, d'une exécution à une autre, les nombres générés soient les mêmes. La dernière instruction de ce programme appelle tout simplement la fonction `print_random_list` en évoquant son nom suivi des parenthèses vides `()`.

L'exécution de ce programme affiche le résultat suivant :

```
rd_list=[3, 10, 2, 5, 2]
```

Remarque

Comme mentionné précédemment, si vous exécutez ce programme plusieurs fois, vous obtenez les mêmes valeurs de la liste `rd_list`, puisque le générateur de nombres aléatoires est initialisé avec la même valeur 1. Si vous supprimez l'instruction `rd.seed(1)` de votre programme ou si vous utilisez un autre nombre que 1 lors de l'appel de cette fonction, par exemple si vous l'appelez avec `rd.seed(2)`, alors il y a de très fortes chances que vous obteniez des valeurs différentes de la liste `rd_list` !

© Editions ENI - All rights reserved

Remarque

Pour générer une liste de cinq nombres aléatoires compris entre 1 et 10, comme dans notre exemple, nous aurions pu utiliser directement la fonction `rd.choices(range(1,11),k=5)` *sans avoir à écrire le code de la fonction* `print_random_list` *! Cependant, le code de notre exemple nous permet d'introduire le concept de fonction de manière pédagogique.*

Lorsque nous développons un programme Python, il est très important de distinguer le programme principal des sous-programmes correspondant aux fonctions qui, elles, sont définies au sein de ce programme principal. Par exemple, dans le code précédent, les trois instructions :

```
import random as rd
...
rd.seed(1)
print_random_list()
```

sont définies au sein du programme principal. Le bloc du code défini à la suite des deux points : situé juste après le nom de la fonction `print_random_list` est appelé le corps de cette fonction et considéré comme un sous-programme du programme principal.

4.6.2 Les fonctions avec paramètres

La fonction `print_random_list` que nous avions définie précédemment n'attend pas d'arguments puisque, lors de sa définition, nous avons laissé les deux parenthèses `()` vides. À chaque fois que cette fonction sera appelée, elle affichera une liste de cinq valeurs aléatoires comprises entre 1 et 10. Justement, la limitation de cette fonction réside dans le fait que, d'une exécution à une autre, le nombre d'éléments dans la liste `rd_list` sera toujours de cinq et chaque élément de cette liste sera toujours compris entre 1 et 10. Cela peut être suffisant pour les besoins d'un programme. Cependant, lorsque nous développons une fonction, il est intéressant de la rendre la plus générique possible afin d'élargir les possibilités de son utilisation ! Par exemple, nous pouvons réécrire la fonction `print_random_list` comme dans le programme suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *fonction-2.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import random as rd

def print_random_list(n, min_val, max_val):

    rd_list = []
    for i in range(n):
        rd_list.append(rd.randint(min_val, max_val))

    print(f"rd_list={rd_list}")

rd.seed(1)
print_random_list(5, 1, 10)
print_random_list(10, 1, 100)
```

Les différences entre ce nouveau programme et le programme précédent sont les suivantes :

- Lors de la déclaration de la fonction `print_random_list`, les parenthèses ne sont plus vides ! Cette fois, nous avons déclaré explicitement que cette fonction attend les trois valeurs `n`, `min_val` et `max_val` pour être exécutée.
- Dans le bloc de code de la fonction `print_random_list`, nous avons utilisé la valeur du paramètre `n` pour fixer le nombre d'itérations de la boucle `for`. Également, nous avons utilisé les valeurs des paramètres `min_val` et `max_val` afin de fixer respectivement les bornes inférieure et supérieure de l'intervalle de définition des valeurs de la liste `rd_list`.
- Nous avons aussi appelé à deux reprises la fonction `print_random_list`. Le premier appel a été effectué avec les arguments 5, 1 et 10 pour reproduire le même comportement de cette fonction que celui de l'exemple précédent et générer une liste de cinq nombres aléatoires compris entre 1 et 10. Le deuxième appel a été effectué avec les arguments 10, 1 et 100 permet de générer une liste de dix nombres aléatoires compris entre 1 et 100.

L'exécution de ce programme affiche le résultat suivant :

```
rd_list=[3, 10, 2, 5, 2]
rd_list=[64, 98, 58, 61, 84, 49, 27, 13, 63, 4]
```

La différence entre la définition de la fonction `print_random_list` de ce dernier programme est clairement plus générique que sa définition dans le programme précédent !

© Editions ENI - All rights reserved

4.6.3 Les valeurs par défaut des paramètres

Lors de la définition de la fonction `print_random_list` précédente, nous avons utilisé des paramètres que l'on appelle des paramètres positionnels. Ils sont positionnels car l'ordre de la transmission des arguments `n`, `min_val` et `max_val` lors de l'appel de la fonction `print_random_list` est important ! Ainsi, dans l'appel `print_random_list(10,1,100)`, la valeur 10 qui correspond à la valeur de l'argument situé à la première position sera associée avec le paramètre `n`, la valeur `1` qui correspond à la valeur de l'argument situé à la deuxième position sera associée avec le paramètre `min_val` et la valeur `100` qui correspond à la valeur de l'argument situé à la troisième position sera associée avec le paramètre `max_val`.

Pour nous débarrasser de la contrainte où l'ordre de la passation des arguments d'une fonction est important, nous pouvons utiliser des paramètres par mot-clé lors de la définition d'une fonction, comme dans le code de l'exemple suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *fonction-3.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import random as rd

def print_random_list(n = 5, min_val =1, max_val=100):
    rd_list = []
    for i in range(n):
        rd_list.append(rd.randint(min_val, max_val))
    print(f"rd_list={rd_list}")

rd.seed(1)
print_random_list(10, 100, 200)
print_random_list()
print_random_list(min_val = -100, max_val=100, n=4)
print_random_list(min_val = 50, max_val=55)
print_random_list(n=6)
```

Dans ce nouveau code, le corps de la fonction `print_random_list` est identique par rapport au corps de cette même fonction de l'exemple précédent. En revanche, cette fois nous avons précisé des valeurs par défaut pour les paramètres `n`, `min_val` et `max_val` et qui sont respectivement les valeurs `5`, `1` et `100`.

Affecter des valeurs par défaut aux paramètres d'une fonction a deux impacts sur l'utilisation de cette fonction : le premier impact est que lors de l'appel à cette fonction, un paramètre qui a une valeur par défaut devient facultatif. Le deuxième impact est que l'ordre du passage des arguments à cette fonction peut ne pas correspondre à l'ordre dans lequel les paramètres ont été définis. Ces deux impacts sont illustrés par les cinq appels de la fonction `print_random_list` à la fin du code de ce dernier exemple et qui ont la signification suivante :

- Dans l'appel `print_random_list(10, 100, 200)`, le paramètre `n` sera égal à 10, le paramètre `min_val` sera égal à 100 et le paramètre `max_val` sera égal à 200.
- Dans l'appel `print_random_list()`, les trois paramètres `n`, `min_val` et `max_val` vont prendre leurs valeurs par défaut ! C'est-à-dire que le paramètre `n` sera égal à 5, le paramètre `min_val` sera égal à 1 et le paramètre `max_val` sera égal à 100.
- Dans l'appel `print_random_list(min_val = -100, max_val=100, n=4)`, le paramètre `n` sera égal à 4, le paramètre `min_val` sera égal à -100 et le paramètre `max_val` sera égal à 100.
- Dans l'appel `print_random_list(min_val = 50, max_val=55)`, le paramètre `n` sera égal à sa valeur par défaut est qui est égal à 5, le paramètre `min_val` sera égal à 50 et le paramètre `max_val` sera égal à 55.
- Dans l'appel `print_random_list(n=6)`, le paramètre `n` sera égal à 6 et les paramètres `min_val` et `max_val` seront égaux à leur valeur par défaut qui sont respectivement 1 et 100.

Le code de cet exemple affiche les résultats suivants :

```
rd_list=[117, 172, 197, 108, 132, 115, 163, 197, 157, 160]
rd_list=[84, 49, 27, 13, 63]
rd_list=[-93, -1, 10, 55]
rd_list=[50, 55, 53, 52, 55]
rd_list=[30, 76, 14, 41, 4, 3]
```

En règle générale, dès qu'un paramètre est associé à une valeur par défaut, ce paramètre devient un paramètre par mot-clé et donc un paramètre facultatif !

© Editions ENI - All rights reserved

Si l'appel à une fonction ne contient pas de valeur pour un argument correspondant à un paramètre par mot-clé, alors c'est la valeur par défaut de ce paramètre qui sera utilisée.

Comme nous l'avons vu, une instruction qui permet de définir une fonction commence par le mot-clé `def` suivi du nom de la fonction à définir, puis le nom de cette fonction est suivi de la liste des paramètres de cette fonction qui sont indiqués entre parenthèses et le caractère : se positionne à la fin de cette instruction pour indiquer le début du corps de cette fonction. Dans une telle instruction, le nom de la fonction ainsi que les paramètres indiqués entre parenthèses sont appelés la signature de cette fonction.

Dans la signature d'une fonction, nous pouvons utiliser des paramètres positionnels en même temps que des paramètres par mot-clé. Dans ce cas-là, il faut toujours commencer par la déclaration des paramètres positionnels suivis par les paramètres par mot-clé comme dans l'exemple suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *fonction-4.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import random as rd

def print_random_list(n, min_val =1, max_val=100):
    rd_list = []
    for i in range(n):
        rd_list.append(rd.randint(min_val, max_val))
    print(f"rd_list={rd_list}")

rd.seed(1)
print_random_list(5)
print_random_list(5, min_val= 90)
```

Dans cet exemple, le paramètre n est un paramètre positionnel, puisqu'il n'a pas de valeur par défaut, alors que les deux autres paramètres `min_val` et `max_val` sont des paramètres par mot-clé. De ce fait, lors de l'appel à la fonction `print_random_list`, le premier argument est obligatoire et il correspondra toujours au paramètre n, alors que les arguments `min_val` et `max_val` sont facultatifs, comme le montrent les deux dernières instructions de notre exemple.

Ce programme affiche le résultat suivant :

```
rd_list=[18, 73, 98, 9, 33]
rd_list=[91, 97, 97, 97, 100]
```

Le premier appel `print_random_list(5)` mentionne l'argument obligatoire correspondant au paramètre positionnel n, mais ne mentionne pas les arguments correspondant aux deux autres paramètres `min_val` et `max_val` qui sont des paramètres par mot-clé et donc facultatifs. Également, le deuxième appel `print_random_list(5, min_val= 90)` mentionne l'argument obligatoire correspondant au paramètre n, mais uniquement l'argument correspondant au paramètre `min_val` qui est mentionné.

Si, lors de l'appel à une fonction définie avec des paramètres positionnels et des paramètres par mot-clé, nous commençons par mentionner les arguments correspondant à un des paramètres par mot-clé, alors l'interpréteur Python nous signale une erreur. Par exemple, une erreur est affichée si vous remplacez la dernière instruction du dernier exemple par l'instruction suivante :

```
print_random_list(min_val= 90, 5)
```

En résumé, lors de la déclaration d'une fonction, il faut toujours commencer par la déclaration des paramètres positionnels suivis de la déclaration des paramètres par mot-clé, et lors de l'appel à une telle fonction, il faut toujours commencer par les arguments correspondant aux paramètres positionnels en respectant l'ordre de leur déclaration suivis par les arguments correspondant aux paramètres par mot-clé pour lesquels l'ordre n'est pas important.

4.6.4 Renvoi de résultats

Une fonction peut renvoyer un résultat à la suite de son exécution, comme le montre l'exemple suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *fonction-4.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
import random as rd

def get_random_list(n, min_val =1, max_val=100):
    rd_list = []
    for i in range(n):
```

© Editions ENI - All rights reserved

```
        rd_list.append(rd.randint(min_val, max_val))

    return rd_list

rd.seed(1)

my_list = print_random_list(10, 1,100)
print(f"rd_list={my_list}")
```

Ce nouveau code reprend l'idée de l'exemple précédent, mais cette fois nous avons supprimé l'instruction d'affichage de la liste `rd_list` qui était définie dans le bloc de code de la fonction `print_random_list` du programme précédent et nous avons ajouté dans ce bloc de code une instruction particulière qui est l'instruction `return rd_list`. Cette dernière instruction permet de retourner la liste `rd_list` à chaque fois que la fonction `get_random_list` est appelée. Dans le programme principal, la liste `my_list` reçoit le résultat de l'exécution de la fonction `get_random_list`, puis la liste `my_list` est affichée avec la dernière instruction. Ce programme affiche le résultat suivant :

```
rd_list=[18, 73, 98, 9, 33, 16, 64, 98, 58, 61]
```

Dans cet exemple, la fonction `get_random_list` a renvoyé une liste, mais de façon générale, nous pouvons renvoyer n'importe quelle structure de données à l'aide de l'instruction `return`.

4.6.5 La portée des variables

Les variables définies au sein d'un bloc de code d'une fonction sont appelées des variables locales contrairement aux variables définies au niveau du programme principal qui, elles, sont appelées des variables globales. Pour mieux comprendre les notions de variables locales et de variables globales, examinons le programme suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *local-var.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
n=10

def my_function():
    n=2
```

```
    print(f"Dans la fonction my_function n= {n}")

print(f"Dans le programme principal n= {n}")
my_function()
print(f"Dans le programme principal n= {n}")
```

Ce programme commence par la déclaration d'une variable globale n=10 qui est située au niveau du programme principal. La fonction my_function définit elle aussi une variable n=2. Cette dernière variable est locale puisqu'elle est définie au sein d'un sous-programme my_function !

L'exécution de ce programme affiche le résultat suivant :

```
Dans le programme principal n= 10
Dans la fonction my_function n= 2
Dans le programme principal n= 10
```

La première ligne de ces résultats montre la valeur de la variable n avant l'appel à la fonction my_function.

La deuxième ligne correspond au résultat de print de la fonction my_function. Le résultat de cette première ligne montre que la variable n est égale à 2 et c'est sans surprise, puisque nous lui avons explicitement affecté la valeur 2 en début du bloc de code de la fonction my_function. En revanche, la troisième ligne, qui correspond au résultat de la fonction print située au sein du programme principal, montre que la variable n est égale à 10 ! En effet, l'affectation n=2 dans la fonction my_function n'a aucun effet sur la variable n du programme principal ! En réalité, lors de l'exécution de l'instruction n=2 de la fonction my_function, il y avait simultanément dans la mémoire de l'ordinateur deux variables distinctes et qui portent le même nom n. En d'autres termes, la variable n de la fonction my_function est une variable locale à cette fonction et elle occupe des bits mémoire différents des bits mémoires occupés par la variable globale n du programme principal.

Plus encore, si vous remplacez l'instruction n=2 par n=n+2 dans la fonction my_function, vous remarquerez que le programme signale une erreur à l'exécution car, bien que la variable n existe dans le programme principal, lors de l'évaluation de l'expression n+2 au sein de cette fonction my_function, l'interpréteur Python ne connaît pas encore la variable n et donc il ne pourra pas réaliser cette opération d'addition.

© Editions ENI - All rights reserved

En synthèse, une fois dans une fonction, l'interpréteur Python ne reconnaît plus les variables définies au sein du programme principal.

Dans la pratique, nous avons souvent besoin que des fonctions utilisent des variables globales. Pour cela, il faut utiliser le mot-clé `global`, comme le montre l'exemple du code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *global-var.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
n=10

def my_function():
    global n
    n=2
    print(f"Dans la fonction my_function n= {n}")


print(f"Dans le programme principal n= {n}")
my_function()

print(f"Dans le programme principal n= {n}")
```

Ce programme est quasiment le même que le programme précédent, mais cette fois nous avons utilisé l'instruction `global n` au début de la fonction `my_function`. En mentionnant au sein de la fonction `my_function` que la variable `n` est globale, au moment de l'exécution de l'instruction `n=2`, l'interpréteur Python ne va pas créer une deuxième variable, mais il va plutôt utiliser la variable `n` déjà définie au sein du programme principal. Dans ce cas, toute modification de la variable `n` au sein de la fonction `my_function` persistera à la fin de l'exécution de cette fonction. Ainsi, l'exécution de ce nouveau code affiche le résultat suivant :

```
Dans le programme principal n= 10
Dans la fonction my_function n= 2
Dans le programme principal n= 2
```

On voit bien que cette fois la variable `n` qui était égale à 10 dans le programme principal avant l'appel de la fonction `my_function` a été modifiée après l'exécution de cette fonction.

4.6.6 Passage d'arguments à une fonction

Avec le langage Python, lorsqu'une fonction est appelée, ses paramètres sont affectés avec les références mémoire des arguments utilisés lors de cet appel.

Attention, pour ceux qui ont l'habitude de travailler avec des langages tels que C, C++ ou Java, avec le langage Python on ne peut pas passer des arguments par valeurs. Tous les arguments seront passés par référence. De plus, toute modification dans une fonction d'une variable de type immuable, tel que le type `int` ou le type `str`, ne sera pas persistante après l'exécution de cette fonction. En effet, lorsque nous modifions la valeur d'un objet de type immuable, c'est un autre objet avec une autre référence mémoire qui sera réellement créé. Pour mieux comprendre le mécanisme du passage des arguments à une fonction, examinons le code suivant :

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *fonction-5.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
def change_param(int_param, str_param, list_param):

    int_param=10
    str_param = "Un trou de ver n'est pas forcément transparent"
    list_param.append(10)

    print(f"Dans change_param
:\n\tint_param={int_param}\n\tstr_param={str_param}\n\tlist_param
={list_param}\n")


my_int=1
my_str="Un trou noir c'est troublant"
my_list=[1,2,3]

print(f"Dans le programme principal:
\n\tmy_int={my_int}\n\tmy_str={my_str}\n\tmy_list={my_list}\n")

change_param(my_int, my_str, my_list)

print(f"Dans le programme principal:
\n\tmy_int={my_int},\n\tmy_str={my_str}\n\tmy_list={my_list}\n")
```

© Editions ENI - All rights reserved

Dans le programme principal, nous avons déclaré les trois variables suivantes :

- La variable `my_int` de type `int` qui est donc une variable immuable.
- La variable `my_str` de type `str` qui est donc une variable immuable.
- La variable `my_list` de type `list` qui est donc une variable modifiable.

Puis nous avons affiché les valeurs de ces trois variables. Ensuite, nous avons appelé la fonction `change_param` en lui fournissant ces trois variables en arguments. De retour au programme principal, nous avons de nouveau affiché les valeurs de ces trois variables.

La fonction `change_param` modifie les valeurs de ses paramètres puis affiche ces nouvelles valeurs.

L'exécution de ce nouveau programme affiche le résultat suivant :

```
Dans le programme principal:
       my_int=1
       my_str=Un trou noir c'est troublant
       my_list=[1, 2, 3]

Dans change_param :
       int_param=10
       str_param=Un trou de ver n'est pas forcément transparent
       list_param=[1, 2, 3, 10]

Dans le programme principal:
       my_int=1,
       my_str=Un trou noir c'est troublant
       my_list=[1, 2, 3, 10]
```

Ces résultats montrent qu'il n'y a que la variable `my_list` pour laquelle la modification dans la fonction `change_param` a persisté après l'exécution de cette fonction. Cela correspond au résultat attendu, puisqu'il n'y a que cette variable qui est d'un type modifiable. Les modifications sur les deux autres variables, `my_int` et `my_str`, n'ont pas persisté après l'exécution de la fonction `change_param`, car elles sont de type immuable.

Dans la fonction `change_param`, lorsque l'affectation `int_param=10` a été réalisée, l'interpréteur Python a réellement créé un autre objet différent de l'objet dont la référence a été reçue en paramètre. Idem lors de l'affectation de la variable `str_pram` dans la fonction `change_param`.

Si nous voulons persister dans le programme appelant des modifications d'une fonction sur un objet de type immuable, il est obligatoire de retourner la valeur de cet objet avec l'instruction `return` dans la fonction qui réalise la modification. Dans ce cas, de retour au programme appelant, il suffit de recevoir la valeur retournée par cette fonction dans l'objet pour lequel nous souhaitons persister ces modifications, comme dans l'exemple de code suivant :

- Créez un fichier avec l'extension *.py* et saïsissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *fonction-6.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
def change_param(int_param, str_param, list_param):

    int_param=10
    str_param = "Un trou de ver n'est pas forcément transparent"
    list_param.append(10)

    print(f"Dans change_param :
\n\tint_param={int_param}\n\tstr_param={str_param}\n\tlist_param=
{list_param}\n")

    return (int_param, str_param)


my_int=1
my_str="Un trou noir c'est troublant"
my_list=[1,2,3]

print(f"Dans le programme principal
:\n\tmy_int={my_int}\n\tmy_str={my_str}\n\tmy_list={my_list}\n")


my_int,my_str=change_param(my_int, my_str, my_list)


print(f"Dans le programme principal
:\n\tmy_int={my_int},\n\tmy_str={my_str}\n\tmy_list={my_list}\n")
```

© Editions ENI - All rights reserved

Ce programme est similaire au programme précédent, mais cette fois nous avons ajouté l'instruction `return (int_param, str_param)` à la fin de la fonction `change_param` et nous avons reçu le résultat de cette fonction dans les variables `my_int` et `my_str` avec l'instruction :

```
my_int,my_str=change_param(my_int, my_str, my_list)
```

Le résultat de l'exécution de ce programme est le suivant :

```
Dans le programme principal:
        my_int=1
        my_str=Un trou noir c'est troublant
        my_list=[1, 2, 3]

Dans change_param :
        int_param=10
        str_param=Un trou de ver n'est pas forcément transparent
        list_param=[1, 2, 3, 10]

Dans le programme principal:
        my_int=10,
        my_str=Un trou de ver n'est pas forcément transparent
        my_list=[1, 2, 3, 10]
```

Cette fois, toutes les modifications au sein de la fonction `change_param` ont persisté dans le programme principal.

Remarque

Dans la mesure du possible, éviter les effets de bord des fonctions est une bonne pratique, car cela peut rendre l'analyse d'un code plus complexe.

4.6.7 Les fonctions récursives

Nous avons vu qu'une fonction est tout simplement un sous-programme qui peut être appelé de n'importe quel endroit d'un programme à partir du moment où cette fonction a été définie.

Une fonction peut également appeler d'autres fonctions, voire s'appeler elle-même ! Lorsqu'une fonction fait des appels à elle-même, elle est dite récursive. L'exemple du code suivant montre une fonction récursive qui permet de calculer la factorielle d'un nombre entier reçu en paramètre.

- Créez un fichier avec l'extension *.py* et saisissez dans ce nouveau fichier le code suivant, puis exécutez ce code. Vous pouvez également utiliser le fichier *rec_function.py* situé dans le dossier *Code\Bases* de ce chapitre.

```
def factorial(n):
    if n<2:
        return 1
    else:
        return n*factorial(n-1)

print(f"4!={factorial(4)}")
```

La fonction `factorial` est une fonction récursive puisque nous trouvons un appel à elle-même au sein de son propre bloc de code. Cette fonction retourne la valeur 1 si la valeur de son paramètre est inférieure à 2 ; sinon, si la valeur de son paramètre est supérieure à 2, elle calcule l'expressionn`*factorial(n-1)`. Dans cette dernière expression, l'appel récursif à la fonction `factorial` a été réalisé en utilisant un argument égal à la valeur du paramètre n moins 1 !

Afin de mieux comprendre la notion de fonction récursive, nous allons dérouler notre programme étape par étape pour calculer `4!`. Sachant que `4! = 4*3*2*1 =24`, voici les étapes du déroulement de notre programme :

1. La première étape correspond au premier appel à la fonction `factorial` avec la valeur 4 au niveau de la dernière ligne du programme principal. À cet instant, l'interpréteur Python alloue un espace mémoire pour l'exécution de la fonction `factorial` en initialisant son unique paramètre n avec la valeur 4.
2. À cette deuxième étape, l'interpréteur Python exécute le bloc du code de la fonction `factorial` ; il s'agit de la première exécution de ce bloc de code. À ce stade, la condition `if 4<2` échoue et c'est donc le bloc associé à la clause `else` qui est exécuté et qui est sous la forme :

```
return 4 * factorial(3)
```

© Editions ENI - All rights reserved

À ce stade, l'interpréteur Python doit d'abord exécuter `factorial(3)` avant de finaliser l'exécution de cette dernière instruction. De ce fait, il sauvegarde le contexte de l'exécution de la fonction `factorial(4)` et il appelle une fonction `factorial` en lui fournissant l'argument 3. En effet, pour l'interpréteur Python, l'appel récursif à la fonction `factorial` sera traité comme tout autre appel de fonction non récursif !

Donc, on se rappellera qu'à l'étape 2, il y a bien une fonction qui attend le résultat de `factorial(3)` avant de calculer et retourner `4 * factorial(3)`.

3. À cette troisième étape, l'interpréteur Python procède à l'exécution de la fonction `factorial(3)`. Il alloue un espace mémoire pour l'exécution de la fonction `factorial` avec le paramètre n initialisé avec la valeur 3.
4. À cette quatrième étape, l'interpréteur Python exécute le bloc du code de la fonction `factorial` ; il s'agit de la deuxième exécution de ce bloc de code. À ce stade, la condition `if 3<2` échoue, et c'est donc le bloc associé à la clause `else` qui est exécuté et qui est sous la forme :

```
return 3 * factorial(2)
```

À ce stade, l'interpréteur Python doit d'abord exécuter `factorial(2)` avant de finaliser l'exécution de cette dernière instruction. De ce fait, il sauvegarde le contexte de l'exécution de la fonction `factorial(3)` et il appelle une fonction `factorial` en lui fournissant l'argument 2. Comme lors de l'appel récursif précédent, pour l'interpréteur Python, l'appel récursif `factorial(2)` sera traité comme tout autre appel de fonction non récursif !

Donc, on se rappellera qu'à l'étape 4, il y a bien une fonction qui attend le résultat de `factorial(2)` avant de calculer et retourner la valeur de `3 * factorial(2)`.

5. À cette cinquième étape, l'interpréteur Python procède à l'exécution de la fonction `factorial(2)`. Il alloue un espace mémoire pour l'exécution de cette fonction `factorial` avec le paramètre n initialisé avec la valeur 2.

6. À cette sixième étape, l'interpréteur Python exécute le bloc du code de la fonction `factorial` ; il s'agit de la troisième exécution de ce bloc de code. Encore à ce stade, la condition `if 2<2` échoue et c'est donc le bloc associé à la clause `else` qui est exécutée et qui est sous la forme :

```
return 2 * factorial(1)
```

À ce stade, l'interpréteur Python doit d'abord exécuter `factorial(1)` avant de finaliser l'exécution de cette dernière instruction. De ce fait, il sauvegarde le contexte de l'exécution de la fonction `factorial(2)` et il appelle une fonction `factorial` en lui fournissant l'argument `1`. Comme lors de l'appel récursif précédent, pour l'interpréteur Python, l'appel récursif `factorial(1)` sera traité comme tout autre appel de fonction non récursif !

Donc, on se rappellera qu'à l'étape 6, il y a une fonction qui attend le résultat de `factorial(1)` avant de calculer et retourner la valeur de `2 * factorial(1)`.

7. À cette septième étape, l'interpréteur Python procède à l'exécution de la fonction `factorial(1)`. Il alloue un espace mémoire pour l'exécution de cette fonction `factorial` avec le paramètre `n` initialisé avec la valeur `1`.

8. À cette huitième étape, l'interpréteur Python exécute le bloc du code de la fonction `factorial`, il s'agit de la quatrième exécution de ce bloc de code. Cette fois, la condition `if 1<2` réussit et c'est le bloc de la clause `if` qui est exécuté. Donc l'instruction `return 1` est exécutée afin de retourner la valeur `1` à la fonction appelante au niveau de l'étape 6 précédente, et ainsi l'exécution de l'appel `factorial(1)` prend fin. Cette fois, il n'y a pas eu d'appel récursif !

© Editions ENI - All rights reserved

9. De retour à l'étape 6, l'instruction qui était en attente et qui pour rappel est l'instruction suivante :

   ```
   return 2 * factorial(1)
   ```

   se transforme en :

   `return 2 * 1` (car `factorial(1)` a retourné 1)

   Puis l'expression `(2*1)` est évaluée et la valeur résultante de 2 est retournée à la fonction appelante de l'étape 4.

10. De retour à l'étape 4, l'instruction qui était en attente et qui, pour rappel, est l'instruction suivante :

    ```
    return 3 * factorial(2)
    ```

    se transforme en :

    `return 3 * 2` (car `factorial(2)` a retourné 2)

    Puis l'expression `(3*2)` est évaluée et la valeur résultante de 6 est retournée à la fonction appelante de l'étape 2.

11. De retour à l'étape 2, l'instruction qui était en attente et qui, pour rappel, est l'instruction suivante :

    ```
    return 4 * factorial(3)
    ```

    se transforme en :

    `return 4 * 6` (car `factorial(3)` a retourné 6)

    Puis l'expression `(4*6)` est évaluée et la valeur résultante de 24 est retournée à la fonction appelante située au niveau de l'argument de la fonction `print` du programme principal. Cette fonction `print` devrait donc afficher la valeur `24`.

Comme attendu, l'exécution de ce programme donne le résultat suivant :

```
4!=24
```

Lors du développement de fonctions récursives, il faut faire très attention aux critères d'arrêt des appels récursifs. En effet, il faut bien s'assurer que la ou les conditions d'arrêt sont correctement positionnées dans le corps d'une fonction

récursive, au risque de saturer la mémoire de l'ordinateur à cause des appels récursifs infinis ! Dans notre exemple, c'est la condition `if n<2` conjuguée avec les appels récursifs `factorial(n-1)` où nous avons décrémenté la valeur de l'argument `n` qui nous garantissent que nos appels récursifs prennent fin à un moment donné.

Bien sûr, dans notre exemple, si vous tentez d'exécuter la fonction `factorial` avec un nombre inapproprié, comme par exemple un nombre négatif, le programme retourne une valeur erronée. Pour que ce programme fonctionne correctement, quelle que soit la valeur transmise à la fonction `factorial`, il est nécessaire d'ajouter des contrôles sur la valeur du paramètre `n`. Nous avons omis ces contrôles dans l'exemple pour des raisons de clarté afin de nous focaliser sur l'aspect récursif de cette fonction `factorial`.

4.7 Les listes en compréhension

La notion de listes en compréhension (*List Comprehension* en anglais) est une syntaxe qui permet de définir des listes de manière compacte et élégante.

4.7.1 Les origines des listes en compréhension

En réalité, la syntaxe d'une *List Comprehension* est inspirée de celle utilisée en mathématiques pour définir des ensembles. Par exemple, en mathématiques, nous pouvons définir un ensemble comme suit $A = \{\mathcal{F}(x); x \in B\}$.

Avec cette notation, on définit un ensemble A dont les éléments sont le résultat de l'application de la fonction $\mathcal{F}$ sur les éléments de l'ensemble B. En d'autres termes, l'ensemble A est défini à partir de l'ensemble B.

Cette idée consiste à définir une structure de données à partir d'une autre structure qui est utilisée en Python avec les listes en compréhension. En revanche, contrairement aux mathématiques où les éléments d'un ensemble ne sont ni ordonnés ni redondants, les listes en compréhension définissent de simples listes Python dont les éléments sont ordonnés et peuvent être redondants !

© Editions ENI - All rights reserved